Guess Market - Exercise 2

Name: Dan Alushe

ID: 212472864

Email: danalushe15@gmail.com

GitHub: https://github.com/danal15/guess-market

Bonuses implemented

All four bonuses are implemented. Bonuses 1 and 2 start switched off, as required - the controls for them are in the top bar of the window.

- 1 - Skins. Three colour schemes (Light, Dark, High contrast), chosen from the "Skin" box in the top bar. Each one changes the background of the whole screen, the look of the buttons, and the font and size of the labels. The application starts on Light, the default skin.
- 2 - Animations. Three short animations (a fade of the details panel, a slide when a panel is rebuilt, and a pulse on the file path after a successful load). None of them run for more than a second, and they are all off until the "Animations" box in the top bar is ticked.
- The skins cover the dialogs as well, so switching skin changes the whole application and not only the main window.
- 3 - Charts. Both charts the bonus asks for are drawn. Every event's details include a chart of the two option prices against the trades made, for LMSR and order book events alike, and every user's details include a chart of their account balance after each movement of money.
- 4 - Creating a new event. The "Create new event..." button under the users table lets the selected user invent an event and become its market maker. The form asks for the fields that belong to the chosen method, and the new event then behaves like any other one.

How to run

- Keep guess-market.jar, engine.jar and run.bat together in the same folder, then double click run.bat. Nothing else is needed and nothing has to be installed beyond Java itself.
- The exercise is built as two modules and ships as two jars: engine.jar is the system engine, and guess-market.jar is the user interface. The JavaFX runtime sits inside the interface jar, which also names the engine jar in its manifest, so "java -jar guess-market.jar" is the whole command - no module path, no switches, and no folder of libraries to keep alongside.
- Load a market file with the "Load File" button in the top bar. Only exercise 2 files are supported.
- The "Save Progress" button beside it writes the whole market - balances, holdings, trades, resting orders and closed events - to a .gmstate file. The same "Load File" button opens one again: it decides from the name whether it has been given a market file to read or a saved market to restore, so there is only ever one button to press. Saving progress is not one of the bonuses listed for this exercise; it is carried over from exercise 1, where the model classes were already made serialisable.
- A few sample files are included in the sample-files folder.

How the program is used

Money is always shown with a $ so it cannot be mistaken for a number of shares, and columns of numbers can be sorted by value.

The window has two tabs. The Events tab is for looking around: it lists every event with its status, method, commission and account, and it has filters for method, status and commission where each one also has an "All" option. Clicking an event shows its full details - for LMSR the prices, account, trade history and chart, and for an order book both books with their bids, asks and statistics, plus the participants.

Everything you actually do happens in the Users tab, because every action belongs to somebody. Pick a user on the left, then pick one of their events, and the buttons that appear are the ones that user is allowed to press right now: opening an event if they are its market maker and it has not started, closing it if it is running, and buying shares or placing an order while it is active.

Main new classes

Event, LmsrEvent, OrderBookEvent - Event is now abstract and holds what both trading methods share (identity, commission, status, account, market maker, and the open and close protocol). The two subclasses add what is different: b and the trade history for LMSR, and the base value, the two books and minting for the order book.

OrderMatcher - the order book engine. It matches an incoming order against the resting orders on the same option, then tries to mint against a buyer of the opposite option, and finally rests whatever is left.

OrderBook, Order, ExecutedTrade - the resting bids and asks of one option, one instruction, and one executed trade. Orders carry a running sequence number so that two orders at the same price are always matched oldest first.

User, Holding - a user with a balance, and their position inside one event (shares held, what they paid for them, and the commission they paid).

Market - the single object holding all the events and users, and the counter that gives orders their order.

MarketFileLoader - reads the exercise 2 file and does every check on it before anything is loaded.

GMEngine, GuessMarketEngine - the interface the screens talk to, and its only implementation. The screens never touch the engine's own objects, only the small read-only objects it returns.

RootController, EventsTabController, UsersTabController - the three screen controllers, plus EventDetailView and TradeForms which build the parts that both tabs share.

Decisions I made

- All the actions are in the Users tab. Every action needs somebody to perform it, and the exercise says participation is done from the users area, so the Events tab is only for looking.
- When two orders meet at different prices, the one that was already resting in the book gets its own price and the new order gets the better deal. This is what the simulation file does.
- Minting happens when the two prices together reach the base value d, and only after the engine has first tried to match the order normally, since a normal match is cheaper for the buyer.
- Money for a buy order is not put aside when the order rests, only when it is actually filled. This is why a user can end up below zero, which is exactly the case the exercise describes: it is allowed to happen, the user is told, and from then on that user is blocked.
- A user cannot sell shares they do not own.
- Before an order or a purchase is confirmed the screen shows what it will cost, and warns if the account cannot cover it. The same check is used by the engine itself, so the two can never disagree.
- A blocked user cannot do anything at all, including closing an event they are the market maker of.
- Prices are given in whole cents, between 0.01 and d minus 0.01.
- A purchase must come to at least $0.01. In LMSR, once one option has been bought heavily the other falls towards zero, and a purchase of it can work out to a fraction of a cent. Since money is only ever shown and held to the cent, that would hand over shares for nothing while still paying out in full if that option won. Such a purchase is refused, and the screen says so before the buy is confirmed rather than showing a misleading cost of $0.00. This only limits prices below one cent; a genuinely cheap option at $0.02 can still be bought normally, which is the market working as it should.
- The value of a holding is shown using the last traded price, or the middle price when there has been no trade yet, or half the base value when the book is completely empty.
- If the initial investment of an order book event does not divide by d, only whole share pairs are created and the market maker pays only for those.
- Commissions go to the market maker's own account, both the ones taken on every purchase and the one taken from the winners when an event closes.

- When an LMSR event closes, whatever is left in the event account after paying the winners goes back to the market maker, so the account ends at zero.
- The XML is read with the parser that comes with Java, so no extra libraries are needed apart from JavaFX.
- The console interface from exercise 1 was removed, since the graphical interface replaces it. The engine itself was reused and extended.